

Paradigmas de la ejecución

Reporte de práctica: control de LED con delay() y millis()

Placa: Arduino UNO R4 WiFi | Software: Arduino IDE

1. Objetivo

Comparar la ejecución con **delay()** y **millis()** al controlar tres LED con el mismo circuito, e identificar cómo cambia la atención de otras tareas, como el envío de mensajes al monitor serial.

2. Materiales y conexiones

Se utilizó un Arduino UNO R4 WiFi, una protoboard, tres LED (verde, amarillo y rojo), tres resistencias de 330 Ω, cables Dupont, cable USB y una computadora con Arduino IDE.

LED	Salida digital	Conexión
Verde	D9	Resistencia de 330 Ω en serie
Amarillo	D10	Resistencia de 330 Ω en serie
Rojo	D11	Resistencia de 330 Ω en serie

Los cátodos se conectaron a GND común y la placa se alimentó por USB. **El armado se conservó sin cambios en ambas versiones.**

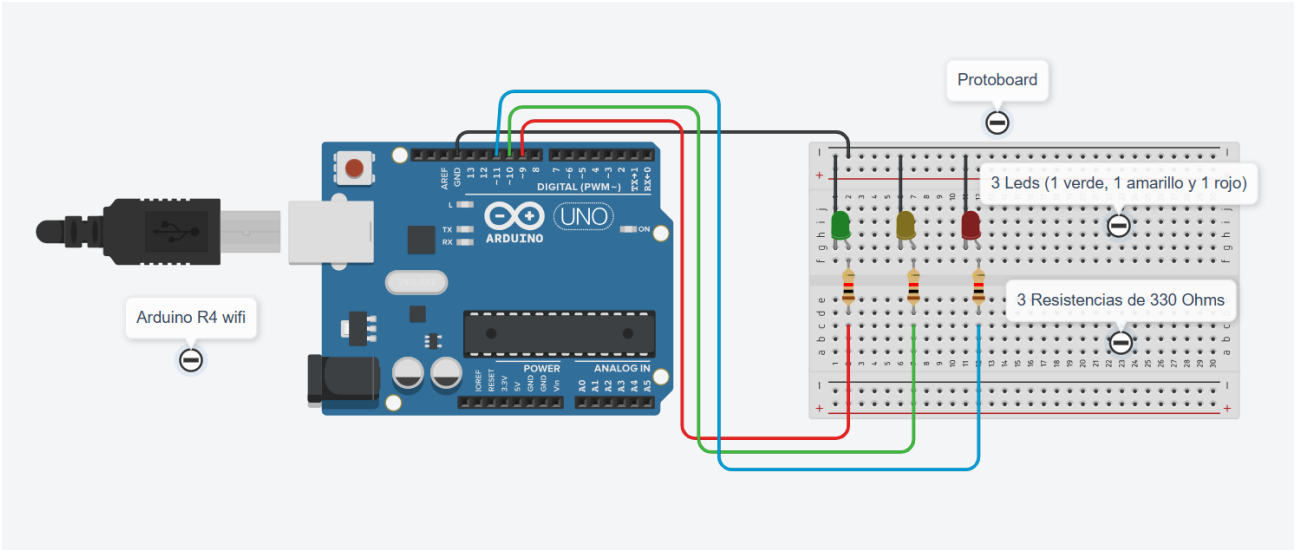


Figura 1. Circuito compartido. El esquema utiliza una placa UNO como referencia; el montaje físico corresponde al Arduino UNO R4 WiFi.

3. Procedimiento

1. Se armó el circuito y se seleccionaron la placa y el puerto en Arduino IDE.
2. Se cargó delay(): verde 500 ms, amarillo 1000 ms y rojo 1500 ms, en secuencia.
3. Con el mismo cableado se cargó la versión con millis(), que revisa por separado el tiempo de cada LED y cambia su estado al cumplirse su intervalo.
4. Para millis() se abrió el monitor serial a 9600 baudios y se verificó la salida del mensaje asociado al encendido del LED rojo.

4. Resultados y comparación

La comparación se basa en los programas ejecutados con el mismo circuito.

Aspecto	Con delay()	Con millis()
Ejecución	Cada espera detiene el avance del programa hasta que termina.	Se consulta el tiempo transcurrido y loop() sigue revisando las tareas.
Encendido	Secuencial: verde, amarillo y rojo. Se mantiene un LED encendido a la vez.	Temporizadores independientes: pueden coincidir varios LED encendidos.
Tiempos de cada LED	Tiempo encendido: Verde: 500 ms. Amarillo: 1000 ms. Rojo: 1500 ms.	Tiempo encendido y también apagado: Verde: 500 ms por estado. Amarillo: 1000 ms por estado. Rojo: 1500 ms por estado.
Ciclo completo	La secuencia suma 3000 ms. Cada LED vuelve a encenderse una vez cada 3 s.	Verde: 1000 ms. Amarillo: 2000 ms. Rojo: 3000 ms.
Monitor serial	No se utilizó en esta versión.	Se configuró a 9600 baudios y envía un mensaje cada vez que enciende el rojo.

Interpretación: en millis(), el intervalo corresponde a un cambio de estado; por eso, un ciclo de encendido y apagado dura el doble. Los tiempos son nominales, calculados del código, no mediciones instrumentales.

La captura muestra ocho líneas de “Ximena la mas chambeadora”. Según el programa, se envían al encender el rojo, aproximadamente cada 3 s, mientras continúa el control de los otros LED.

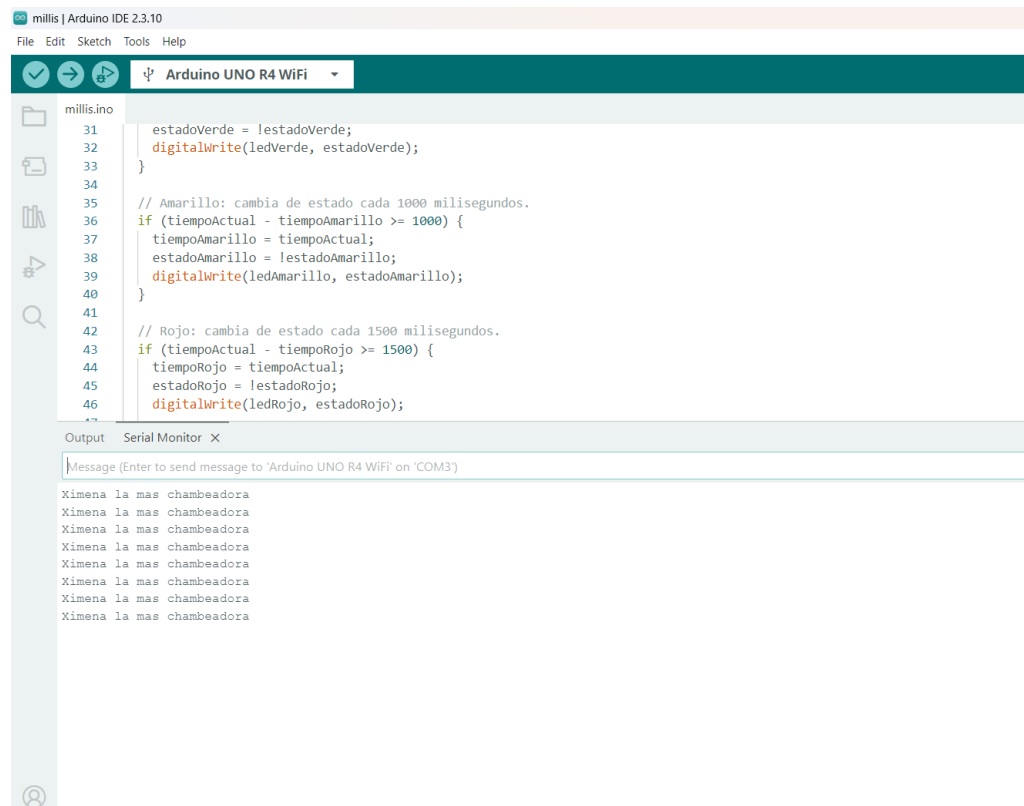


Figura 2. Código y salida del monitor serial de la versión con millis(). La imagen confirma la impresión del texto; no permite medir los intervalos.

5. Evidencias del montaje

Las fotografías muestran el circuito utilizado en las dos prácticas y dos estados de iluminación. El cambio entre `delay()` y `millis()` se realizó únicamente en el programa.

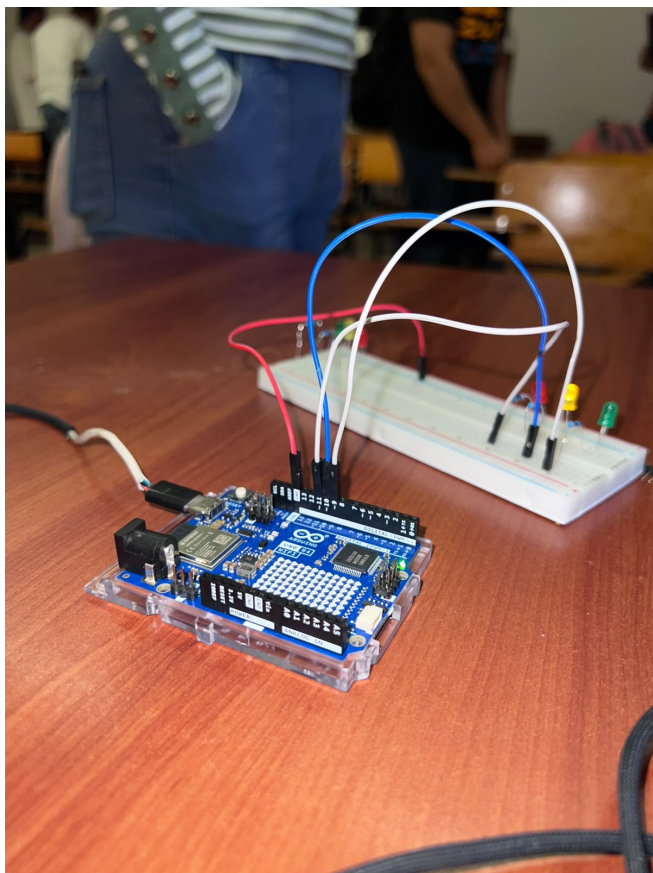


Figura 3. Montaje general y LED amarillo encendido.

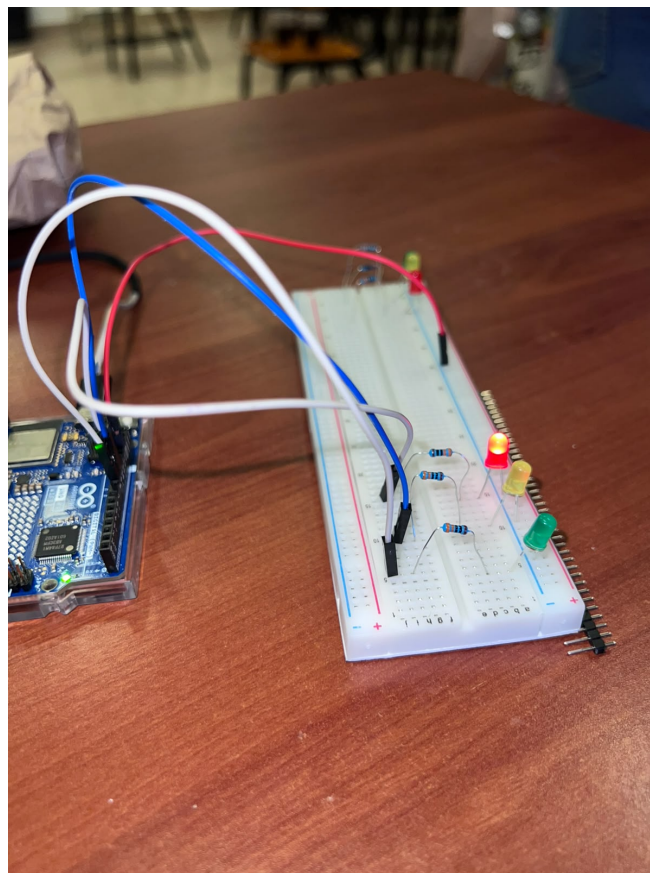


Figura 4. Detalle del circuito y LED rojo encendido.

6. Conclusión

La práctica permitió comparar dos formas de administrar el tiempo sin modificar el circuito. **`delay()`** facilitó una secuencia sencilla, aunque cada espera impidió avanzar a las siguientes instrucciones. **`millis()`** permitió controlar los tres LED con tiempos independientes y atender el envío de mensajes al monitor serial.

La principal diferencia es la organización de la ejecución: con `millis()` el programa puede seguir revisando otras tareas mientras transcurre cada intervalo. La comunicación serial también puede usarse con `delay()`, pero en esa versión las pausas retrasan la ejecución de nuevas instrucciones.